

Dadu-RBD: Robot Rigid Body Dynamics Accelerator with Multifunctional Pipelines

Yuxin Yang

yangyuxin18g@ict.ac.cn

Institute of Computing Technology,
Chinese Academy of Sciences
University of Chinese Academy of
Sciences
Beijing, China

Xiaoming Chen*

chenxiaoming@ict.ac.cn

Institute of Computing Technology,
Chinese Academy of Sciences
Beijing, China

Yinhe Han*

yinhes@ict.ac.cn

Institute of Computing Technology,
Chinese Academy of Sciences
Beijing, China

ABSTRACT

Rigid body dynamics is a core technology in the robotics field. In trajectory optimization and model predictive control algorithms, there are usually a large number of rigid body dynamics computing tasks. Using CPUs to process these tasks consumes a lot of time, which will affect the real-time performance of robots. To this end, we propose a multifunctional robot rigid body dynamics accelerator, named Dadu-RBD, to address the performance bottleneck. By analyzing different functions commonly used in robot dynamics calculations, we summarize their relationships and characteristics, then optimize them according to the hardware. Based on this, Dadu-RBD can fully reuse common hardware modules when processing different computing tasks. By dynamically switching the dataflow path, Dadu-RBD can accelerate various dynamics functions without reconfiguring the hardware. We design the Round-Trip Pipeline and Structure-Adaptive Pipelines for Dadu-RBD, which can greatly improve the throughput of the accelerator. Robots with different structures and parameters can be optimized specifically. Compared with the state-of-the-art CPU, GPU dynamics libraries and FPGA accelerator, Dadu-RBD can significantly improve the performance.

CCS CONCEPTS

• **Computer systems organization** → **Data flow architectures; Reconfigurable computing**; • **Hardware** → **Application specific processors**.

KEYWORDS

Accelerator, Rigid Body Dynamics, Robotics, Pipeline, Dataflow, Multifunctional

ACM Reference Format:

Yuxin Yang, Xiaoming Chen, and Yinhe Han. 2023. Dadu-RBD: Robot Rigid Body Dynamics Accelerator with Multifunctional Pipelines. In *56th Annual*

*Corresponding author.



This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivs International 4.0 License.

MICRO '23, October 28–November 01, 2023, Toronto, ON, Canada
© 2023 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-0329-4/23/10.
<https://doi.org/10.1145/3613424.3614298>

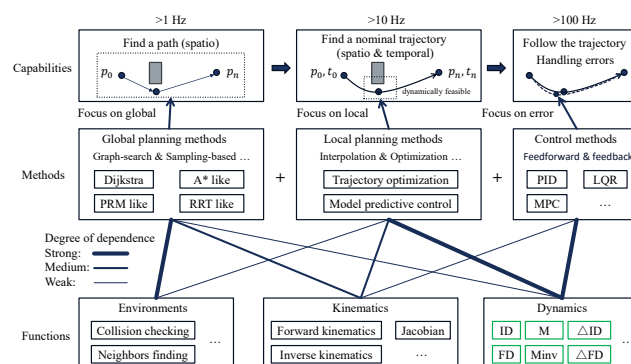


Figure 1: Planning and control framework in robotics.

IEEE/ACM International Symposium on Microarchitecture (MICRO '23), October 28–November 01, 2023, Toronto, ON, Canada. ACM, New York, NY, USA, 13 pages. <https://doi.org/10.1145/3613424.3614298>

1 INTRODUCTION

Robotics is in a stage of rapid development. As demands continue to increase, robots need to face more complex and dynamic environments. Various robot structures also require different methods to ensure the balance of the robot itself and the ability to interact with the environment. Planning and control algorithms are the key technologies to ensure the robot's ability to move.

Traditionally, the planning and control framework is shown in Fig. 1. The planning algorithms are divided into two parts: global planning methods and local planning methods. The global planning methods mainly focus on the overall spatial path, while the local planning methods mainly focus on the spatio-temporal trajectory in a small area. They form the front-end and back-end of the planning algorithms. The front-end can provide a reliable initial value for the optimization algorithm in the back-end.

There are many different global planning methods, such as A*, PRM(*), RRT(*), and a large number of improved methods derived from them. The main purpose of these algorithms is to find a feasible path without obstacles in the space. Therefore they strongly rely on environment-related basic functions such as collision detection and nearest neighbor search. With the continuous improvement of robot performance requirements, hardware acceleration work related to global planning methods continues to emerge, such as [4, 17, 21, 28, 32, 51].

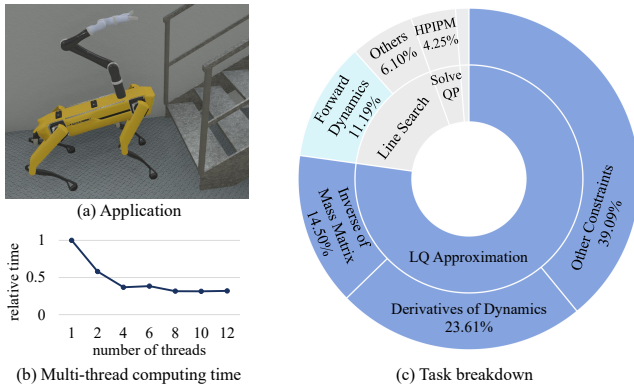


Figure 2: Robot application and performance analysis.

Trajectory optimization (TO) and model predictive control (MPC) [14, 35, 46] are two important types of local planning methods. The MPC algorithm can also be used as a control method. The difference is that the MPC algorithm in planning method will output a nominal trajectory for later following, while the MPC algorithm in control method will output the underlying execution instructions directly, which requires higher performance ($>100\text{Hz}$) [7, 52]. The TO and MPC algorithms contain a large number of rigid body dynamics function calls, which will seriously affect the real-time performance of robots [34, 35]. Depending on the complexity of the robot's structure, the time of the rigid body dynamics computing can account for 30%-90% of the total algorithm running time [6, 35, 39]. To meet the real-time requirements, some existing works accelerate the TO and MPC algorithms through multi-threaded CPUs [11, 19, 25]. There are also some works exploiting the parallelism of GPUs to accelerate these algorithms [2, 3, 16, 18, 37].

In fact, most algorithms in robot dynamics has forward and backward propagation, similar to the training algorithm of neural networks. It is a cache-unfriendly calculation, and require a lot of memory access [6, 12]. When using CPU multithreading for acceleration, memory bottlenecks will be encountered. GPU also has this problem. And the single-task latency of the GPU is relatively large, which is not a good choice for some tasks. In addition, high-performance CPUs or GPUs generally have high power consumption, so they are not very suitable for battery-dependent mobile robots.

To verify the above feature, we build a robot application example (Fig. 2a) and test it in the robot simulator Webots [48] with the control framework OCS2 [10]. As shown in Fig. 2b, when the number of threads increases to a certain level, the performance is no longer significantly improved. On the other hand, from Fig. 2c, we can observe that the proportion of parallelizable parts (LQ Approximation, dark blue) is large and contains various types of tasks. This implies that the entire task has the potential to be further accelerated in parallel.

For energy-efficient real-time computing, there are many end-to-end domain-specific hardware accelerators designed for the MPC algorithms, such as [23, 27, 31, 38, 43, 49, 50]. But they can only deal with linear models or simple robot structures, and cannot handle dynamics models for robots with high degrees-of-freedom (DOF).

Robomorphic [34] tries to use FPGA/ASIC to accelerate rigid body dynamics derivatives. But it only supports this single function (forward dynamics derivatives) in the dynamics calculations, and the throughput is not high. In addition, it also require the cooperation of the CPU. This will bring a lot of communication overhead and extra computational overhead to the CPU.

In response to the above problems, we design a multifunctional robot dynamics accelerator, Dadu-RBD. It is also a general rigid body dynamics accelerator design framework that can be applied to a wide variety of robots. Dadu-RBD needs to solve three design challenges. The first is how to support multifunction related to rigid body dynamics. The second is how to achieve high performance under memory bottlenecks. The third is how to reduce the resource usage when dealing with different robot models.

For the first challenge, the most direct method is to design a separate module for each function, but such a waste of resources is unacceptable. We summarize the relationships of commonly used dynamics algorithms, and analyzes the characteristics of them. On this basis, we carry out hardware-specific optimizations for these algorithms with the goal of minimizing resource usage while ensuring that there is no major loss in performance. At the same time, we designed a global data-driven state machine for the entire Dadu-RBD architecture, so that Dadu-RBD can dynamically switch the dataflow and output results according to the function type, without the need for external instruction sets to control. This allows Dadu-RBD to perform calculations independently of the CPU, which can greatly reduce the communication overhead and further improve the performance of the entire system.

To deal with the second challenge, we design the Round-Trip Pipeline (RTP). It is a medium-grained pipeline at joints level, which has deep pipeline stages that support massive parallelism. Previous work [34] lacked this pipeline depth. They use two big cores to process the forward propagation and the backward propagation respectively. Each core needs to have the general ability to handle all joints, so it cannot be completely optimized for sparsity and takes up more resources. Instead we use as few resources as possible in each submodule for a joint. This strategy can generate a dataflow pattern similar to a systolic array, allowing data transmission and computing time to overlap each other, thereby greatly improving throughput while keeping resource consumption low at the expense of a small increase in latency. RTP can naturally use the FIFO buffer to store the temporary data required by the pipeline, thereby avoiding the large amount of data exchange between the cache and the main memory in the traditional architecture, and reducing the memory access bandwidth pressure.

Whether the third challenge can be solved is the key to the practicality of the whole architecture. We design the Structure-Adaptive Pipelines (SAPs) for Dadu-RBD. It uses the medium-grained pipeline strategy mentioned above, and can be optimized according to the structure of the robot itself. By utilizing the software and hardware co-optimization, we implement resource reuse and calculation sparsity according to certain strategies, which can greatly reduce resource usage while ensuring throughput.

We evaluate our architecture using the same FPGA chip as that used in Robomorphic [34]. Compared with the existing state-of-the-art CPU[40], GPU[40] and FPGA accelerators[34, 40], Dadu-RBD can achieve 10.3 $\times$, 3.4 $\times$ and 6.3 $\times$ higher throughput, respectively,

in the derivatives of dynamics calculations. Compared with the CPU dynamics library Pinocchio [6] and GPU dynamics library GRiD [41], the performance of Dadu-RBD has also been significantly improved.

Our main contributions are summarized as follows:

- Dadu-RBD implements a high-performance, multifunctional domain-specific hardware accelerator for robot rigid body dynamics.
- We summarize the relationships and characteristics of commonly used dynamics algorithms, and propose hardware-specific optimizations for these algorithms, including a new mass matrix generation/inversion algorithm.
- RTP is proposed to improve the throughput. We implemented all necessary basic dynamics functions using RTP, and carried out corresponding software-hardware co-optimization.
- SAPs are proposed to optimize the performance and resource consumption. Independent pipeline arrays can further perform asynchronous calculations.

2 BACKGROUND

In order to design a general-purpose robot dynamics accelerator, we need to model the robot in a general format. As discussed in [12], we can use a topological tree to describe an open-chains robot. As the example shown in Fig. 3, the robot has 5 limbs connected to the body. Four of them are legs composed of three links, and the remaining one is a robotic arm composed of six links. We can assume that the body of the robot is a floating base that connects to a fixed world coordinate system through a virtual 6-DOF joint. Then each link of the robot can correspond to a joint. We can describe each link and joint in the form of matrices. We assume that the robot has N_B (number of bodies/links) joints and links, with a total of N DOF. Each link has its own mass and rotational inertia, which can be represented by the symmetric inertia matrix $I_i \in \mathbb{R}^{6 \times 6}$. Each joint has a specific type. Types of joints include revolute, prismatic, helical, cylindrical, planar, spherical, 3-DOF translation, 6-DOF joint, etc. Different types of joints have different motion subspaces $S_i \in \mathbb{R}^{6 \times N_i}$, where N_i is the DOF of i th joint. For the most common joints in robots (revolute and prismatic), $N_i = 1$ and S_i are one-hot vectors. The definitions of S_i for other type of joints can be found in [12]. When the joint state q_i is given, we can calculate the pose relationship between the two links connected by the joint, which can be represented by the transformation matrix ${}^iX_{\lambda_i} \in \mathbb{R}^{6 \times 6}$, where λ_i is the parent link's id. The transformation matrix ${}^iX_{\lambda_i}$ has a unique sparsity. Its top right 3×3 elements are always 0. The inertia matrix I_i also has sparsity according to

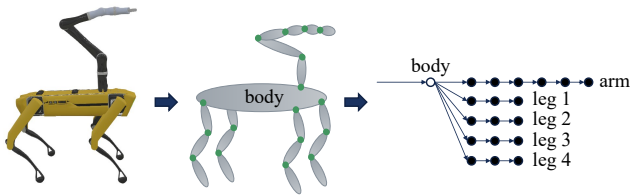


Figure 3: Robot model.

the joint and link. All the above parameters are defined in their respective joint coordinate systems.

For the robots with the same model, N_B , S_i and the sparsity of I_i are all the same. But we may need to calibrate the parameters in I_i and ${}^iX_{\lambda_i}$ for different robots with the same model. For a specific robot, I_i , S_i and parameters in ${}^iX_{\lambda_i}$ can be seen as constant.

Most of calculations in robot dynamics are essentially related to the **equation of motion** for a rigid body system:

$$M(q)\ddot{q} + C(q, \dot{q}, f^{ext}) = \tau, \quad (1)$$

where $q, \dot{q}, \ddot{q}, \tau$ are vectors of position, velocity, acceleration and force/torque variables, respectively, $M(q)$ is the symmetric positive-definite mass matrix (or joint space inertia matrix) of the robot, $C(q, \dot{q}, f^{ext})$ is the generalized bias forces accounts for the Coriolis and centrifugal forces, gravity, and any other forces (f^{ext}) acting on the system other than those in τ .

Different applications require the calculation of the equation of motion (Eq. (1)) from different perspectives:

- Find τ in control algorithms and TO [13, 22, 42, 45, 47];
- Find $\ddot{q}$ in simulation and MPC [10, 15, 22, 24, 35];
- Find M in optimal control and TO [13, 30, 47];
- Find M^{-1} in kinematics and MPC [10, 15, 29, 30];
- Find $\partial_u \tau$ in optimal control and TO [13, 22, 30, 47];
- Find $\partial_u \ddot{q}$ in MPC [8, 10, 15, 35, 36] ($u = [q; \dot{q}]$);

We organize these calculations into different functions in Table 1. Here var u represents both q and $\dot{q}$, and $\partial_u \tau$ means $(\frac{\partial \tau}{\partial q}, \frac{\partial \tau}{\partial \dot{q}})$. The definitions of these functions are consistent with those in [6, 12, 34, 40]. These functions are what Dadu-RBD needs to accelerate.

Table 1: Rigid body dynamics functions overview.

Function Name	Definition
Inverse Dynamics	$\tau = \text{ID}(q, \dot{q}, \ddot{q}, f^{ext})$
Forward Dynamics	$\ddot{q} = \text{FD}(q, \dot{q}, \tau, f^{ext})$
Mass Matrix	$M = M(q)$
Inverse of Mass Matrix	$M^{-1} = \text{Minv}(q)$
Derivatives of ID	$\partial_u \tau = \Delta \text{ID}(q, \dot{q}, \ddot{q}, f^{ext})$
Derivatives of FD	$\partial_u \ddot{q} = \Delta \text{FD}(q, \dot{q}, \tau, f^{ext})$
Derivatives of Dynamics	$\partial_u \ddot{q} = \Delta \text{iFD}(q, \dot{q}, \ddot{q}, M^{-1}, f^{ext})$

3 KEY INSIGHTS

3.1 Relationships in Rigid Body Dynamics

In software algorithms, the different functions of the dynamics are usually independent of each other. This is because different algorithms can be optimized for the desired function, and we can choose the one with the best performance.

However, all functions in Table 1 satisfy the equation of motion (Eq. (1)), there must be a strong relationship between them. Notice that if we fix $q, \dot{q}$, and f^{ext} in Eq. (1), then the whole equation is actually simplified to the form of $\tau = M\ddot{q} + C$, where M, C are both constants. This means that $\ddot{q}$ and τ are two vectors that are linearly related. Once we know one of them, we can easily get another. This relationship holds for both dynamics and gradients of dynamics

[6, 12]. That is to say, we have the following relationships in rigid body dynamics:

$$FD = M^{-1} ID, \quad (2)$$

$$\Delta FD = M^{-1} \Delta ID. \quad (3)$$

The existence of this relationship is very friendly to hardware design, allowing our multifunctional accelerators to reuse the computing resources. The function ID can be calculated using the Recursive Newton-Euler Algorithm (RNEA) [12], and the function ΔID can also be analytically calculated by $\Delta RNEA$ [6]. In software, the function FD is usually calculated using the efficient Articulated Body Algorithm (ABA) [12]. But its gradient calculation is relatively complicated, so we can use the relationship Eq. 3 to get ΔFD from the function ΔID . For further resource sharing, we can completely omit the instantiation of the ABA algorithm, and also use the relationship Eq. 2 to implement the FD function.

There is also a strong relationship between the calculation of the mass matrix and its inverse. There are many ways to compute the inverse of the mass matrix. We can first use the Composite Rigid Body Algorithm (CRBA) [12] to calculate the mass matrix, and then use a certain decomposition algorithm to calculate the inverse of the mass matrix. Because the mass matrix is a symmetric positive definite matrix, it can be decomposed by Cholesky factorization or LDL^T factorization, which can be written as $M = LDL^T$. After the decomposition, we can get M^{-1} efficiently [26].

In many cases, the inverse of the mass matrix is calculated to aid in the calculation of forward dynamics. If we just want to calculate the forward dynamics, we do not even need to know what M^{-1} is. We can transform Eq. (1) into $L^{-T}D^{-1}L^{-1}\ddot{q} = (\tau - C)$, then we can solve $\ddot{q}$ by two back-substitution procedures. But this method has shortcomings. We use the CRBA to first calculate the mass matrix, and then calculate the inverse of the matrix or solve the linear equation system through matrix decomposition. The two serial steps will introduce long latency when accelerated in parallel using dedicated hardware.

In fact, we can do part of the matrix decomposition and back-substitution procedures in advance while computing the M matrix, especially the reciprocal operation. This can overlap the computational latency of the two parts, thereby reducing the latency of the overall computation. This also allows the reuse of resources for computing the mass matrix and its inverse. More details will be discussed in a Section 4.2.

3.2 Characteristics of Rigid Body Dynamics

In order to describe the characteristics of the dynamic algorithm, we first introduce the most basic dynamics algorithm RNEA, as shown in Algorithm 1. We modified it a little in detail to meet the needs of the hardware design, which will be mentioned in Section 4.1. It can be observed that the RNEA algorithm has a serial process of forward propagation and backward propagation. The intermediate results obtained in the forward propagation need to be used in the backward propagation.

This feature also exists for other dynamic algorithms, which can be physically explained as a chain-like dependency between robot joints. We abstract this computing feature, as shown in Fig. 4a. The data transfer between forward pass and back pass is hardware unfriendly. The process of forward pass and backward propagation

Algorithm 1 RNEA

Input: $q, \sin q, \cos q, \dot{q}, \ddot{q}, f^{ext}$

Output: $\tau, [v, a, f]$

```

1:  $v_0 = 0, a_0 = \text{gravity}$ 
2: for  $i = 1 : N_B$  do
3:    ${}^iX_{\lambda_i} = X_i(q_i, \sin q_i, \cos q_i)$ 
4:    $v_i = {}^iX_{\lambda_i} v_{\lambda_i} + S_i \dot{q}_i$ 
5:    $a_i = {}^iX_{\lambda_i} a_{\lambda_i} + S_i \ddot{q}_i + v_i \times S_i \dot{q}_i$ 
6:    $f_i = I_i a_i + v_i \times {}^iI_i v_i - f_i^{ext}$ 
7: for  $i = N_B : 1$  do
8:    $\lambda_i X_i^* = X_i^T(q_i, \sin q_i, \cos q_i)$ 
9:    $\tau_i = S_i^T f_i$ 
10:   $f_{\lambda_i} = \lambda_i X_i^* f_i$ 
```

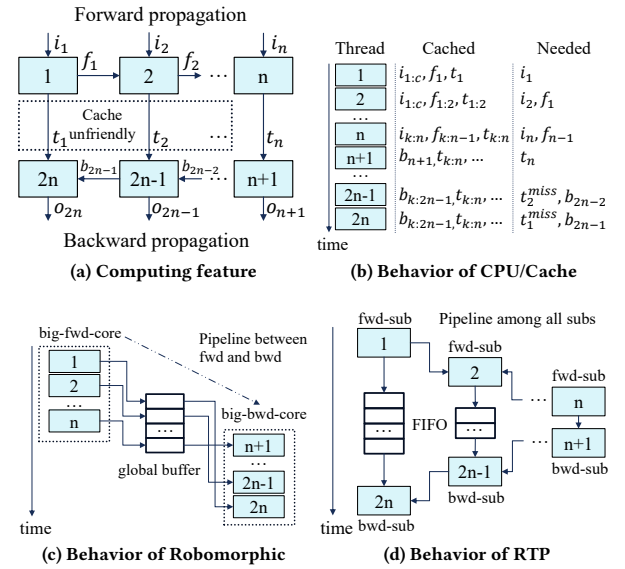


Figure 4: Analyze the behavior of different hardware architectures for the same computing feature.

leads to poor temporal locality of intermediate data. For the modern CPU/Cache architecture, these intermediate results are easily missed in the cache, thus indirectly increasing the memory access pressure, as shown in Fig. 4b.

Actively controlling the on-chip buffer can alleviate this problem, so using hardware such as FPGAs for acceleration is an effective way. Robomorphic [34] focuses on optimizing the latency of calculations, so it makes high-performance computing cores for forward propagation and back propagation respectively, as shown in Fig. 4c. It uses a global on-chip buffer to store intermediate variables. Between the forward core and the backward core, pipelines can be used to improve throughput to a certain extent. These two “big” computing cores are applicable to all joints, so much more resources are required.

In contrast, Dadu-RBD focuses on optimizing the computation throughput. We propose Round-Trip Pipeline to achieve massively pipelined parallelism, as shown in Fig. 4d. The FIFO buffer between the forward and backward propagation acts as a bypass buffer for

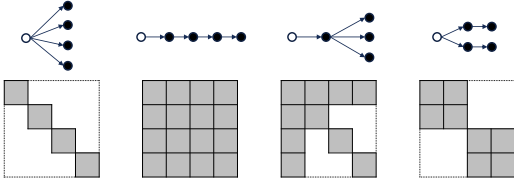


Figure 5: Branch induced sparsity

the pipeline, keeping the entire pipeline running efficiently and reducing the amount of off-chip memory access. Here, each submodule only cares about one joint, and targeted sparsity optimization can be performed on it. At the same time, it is noted that the computing load of different submodules will vary with the depth of the joint, so they can have different optimization strategies. Submodules with heavy loads need to ensure computing latency to maintain the frequency of the overall pipeline; Submodules with light loads can fully reuse computing resources without affecting the overall pipeline frequency.

3.3 Adapte to the robot structure

Robot structures in the real world are diverse, and Dadu-RBD needs to be guided by a general method as a robot dynamics accelerator. We note that the structure of the robot can have a huge impact on the sparsity of the robot dynamics parameter matrix [12] (see Fig. 5). In Algorithm 1, we can also see that the robot structure also has an important impact on the topology of the robot dynamics calculation. These effects are all due to the branching structure of the robot. The intuitive understanding is that the dynamics of a certain joint of the robot will only be propagated to the branches it can affect. After introducing the forward and backward propagation process of the dynamics algorithm, the sparsity complexity of the matrix will be further increased. These sparsity and computational complexity changes due to robot topology are important issues that we must consider.

We address this issue by proposing Structure-Adaptive Pipelines, which optimize the accelerator architecture according to the robot architecture. SAPs contains a set of software and hardware collaborative optimization methods. It will provide effective hardware support according to the branch characteristics of the robot, including time division multiplexing, sparsity optimization, etc. The topology of the robot will also be properly rotated according to the hardware characteristics, so as to reduce the overall resource consumption.

4 ROUND-TRIP PIPELINE DESIGN

4.1 Inverse Dynamics and Its Derivatives

We design the RNEA RTP and the Δ RNEA RTP, as shown in Fig. 6 and Fig. 7.

In the RNEA RTP, each iteration of the loop in the Algorithm. 1 is mapped to a submodule. For example, for a robot arm with $N_B = n$ links, the RNEA RTP has $2n$ submodules, namely R_{fi} and R_{bi} , where $i \in [1, n]$; These submodules are independent of each other and transmit data through FIFO streams. In this way, each submodule can just handle the calculation of the corresponding joint and link.

For the submodule R_{fi} , it needs to update X_i , calculate v_i, a_i, f_i according to the Algorithm 1, then transfer ftr_i forward to R_{fi+1} and dtr_i downward to R_{bi} . For the submodule R_{bi} , it needs to update X_i again, update f_i , compute btr_{i-1} and τ_i , then transfer btr_{i-1} backward to R_{bi-1} and τ_i to $output_i$.

The Δ RNEA RTP also has $2n$ submodules, namely D_{fi} and D_{bi} , where $i \in [1, n]$. It has the same structure as the RNEA RTP, but the input/output and the calculation content in each submodule are different.

For each RNEA or Δ RNEA calculation, it needs to go through forward pass and backward pass. For the shallower (the subscript is smaller) submodules, intermediate data needs to be cached for a long time. Certain bypass buffers are added to our hardware design to avoid pipeline stalls. In this way, the multi-stage pipeline can greatly improve the parallelism of calculations, thereby improving throughput. Although there are multiple similar matrix multiplications inside the submodule, we did not implement a fine-grained matrix multiplication pipeline unit. Doing so will consume a lot of resources and will also make the sparse optimization less efficient.

We have some methods to further optimize the design.

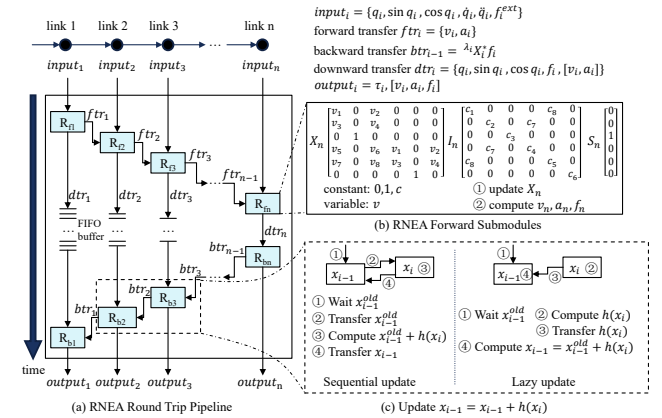
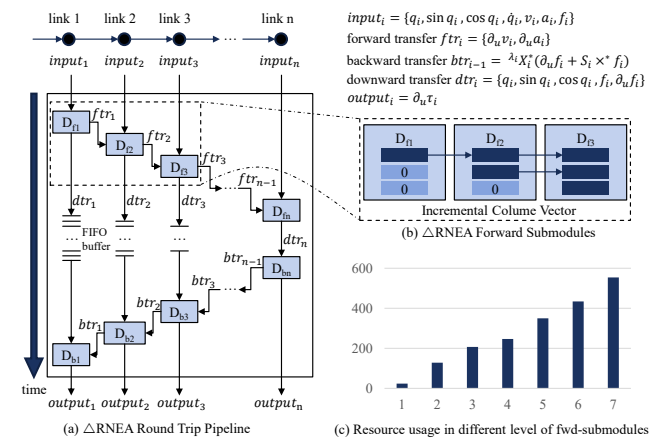


Figure 6: RTP Design for RNEA

Figure 7: RTP Design for Δ RNEA

4.1.1 Sparsity and Constant Optimization. As shown in Fig. 6b, we can optimize the design according to the fixed sparse characteristics and constants in the robot parameters. We assume that the n th joint is the most common revolute joint in robots. Then only 12 elements in the 6×6 matrix X_n are non-constant, and these 12 elements have only 8 different values, all of which are in the form of $c * \sin q$ or $c * \cos q$. The symmetric matrix I_n has only 8 distinct non-zero constants, and the S_n is a one-hot vector. These features are also mentioned in Section 2, which can greatly reduce the computational cost. Therefore, we don't need to design a complete 6×6 matrix vector computing unit, but only need to consider the non-zero part. We can also further optimize the design according to the characteristics of the hardware, such as saving the constants on-chip, or using the look up tables in FPGAs, thereby reducing memory access overhead and improving performance.

4.1.2 Reupdate Transformation Matrix. In the backward submodules R_{bi} and D_{bi} , we reupdate X_i . Instead of buffer and transfer the calculated X_i , we recalculate it. As mentioned above, the calculation cost of x is very small, while the transmission cost is relatively greater. In matrix X_i , only one or two multiplication are required. Since the forward submodules are more complex than the backward submodules, we need to minimize the number of interfaces and tasks of the forward module. Therefore, we choose to transfer less data downward. Only one data q_i is needed for the prismatic joint, and only two data $\sin q_i, \cos q_i$ are needed for the revolute joint.

4.1.3 Lazy Update. There are some operations that need to add or subtract variables from other loop bodies, such as Algorithm 1 line 10, and the corresponding calculation in the Δ RNEA algorithm. If we update the data sequentially as described by the original algorithm, the loopback dependency between loop bodies will destroy the dataflow of the pipeline. Actually we do not need to read them first and then write them back. We can just pass the addend or subtraction to the corresponding submodule, and wait for the next cycle to calculate. We call this approach lazy update, as shown in Fig. 7c.

4.1.4 Incremental Calculation. In the Δ RNEA RTP, the matrices have different sparsity characteristic related to the depth of iterations. In fact, the number of their useful columns is proportional to the iteration depth, as shown in Fig. 7b. We can further utilize column vectors (columns of the matrix) to only pass and calculate useful columns, while each submodule only needs to incrementally initialize newly added columns. This feature will cause deeper submodules to have higher computation load, for which we need to allocate more computing resources to them (Fig. 7c). Due to the increase in calculations, deeper sub-modules will inevitably become the performance bottleneck of the entire RTP, so we can adopt a more aggressive resource reuse strategy for shallower modules to reduce overall resource consumption.

4.2 Mass Matrix and Its Inverse

We propose MMinvGen (Algorithm 2) to generate the mass matrix or the inverse of the mass matrix. It combines the CRBA algorithm [12] and a simplified ABA algorithm [5]. We analyze the differences and similarities of these two algorithms, and further simplify and unify them through software-hardware co-optimization.

Algorithm 2 MMinvGen

Input: $q, \sin q, \cos q, out_M, out_{Minv}$

Output: M or M_{inv}

```

1: for  $i = N_B : 1$  do
2:    ${}^iX_{\lambda_i} = X_i(q_i, \sin q_i, \cos q_i); {}^{\lambda_i}X_i^* = {}^iX_{\lambda_i}^T$ 
3:    $I_i^A \leftarrow I_i; U_i = I_i^A S_i; D_i = S_i^T U_i$ 
4:   if  $out_{Minv}$  then
5:      $M_{inv}[i, i] = D_i^{-1}$ 
6:      $M_{inv}[i, tree_e(i)] = -D_i^{-T} S_i^T F_i[:, tree_e(i)]$ 
7:   if  $out_M$  then
8:      $M[i, i] = D_i$ 
9:      $M[i, tree_e(i)] = F_i^T[:, tree_e(i)] S_{tree_e(i)}$ 
10:  if  $\lambda_i \neq 0$  then
11:    if  $out_{Minv}$  then
12:       $F_i[:, tree_e(i)] \leftarrow U_i M_{inv}[i, tree_e(i)]$ 
13:       $I_i^A \leftarrow U_i D_i^{-1} U_i^T$ 
14:    if  $out_M$  then
15:       $F_i[:, i] = U_i$ 
16:       $F_{\lambda_i}[:, tree_e(i)] \leftarrow {}^{\lambda_i}X_i^* F_i[:, tree_e(i)]$ 
17:       $I_{\lambda_i}^A \leftarrow {}^{\lambda_i}X_i^* I_i^A {}^{\lambda_i}X_{\lambda_i}$ 
18:  if  $out_{Minv}$  then
19:    for  $i = 1 : N_B$  do
20:      if  $\lambda_i \neq 0$  then
21:         $M_{inv}[i, i:] = D_i^{-1} U_i^T {}^iX_{\lambda_i} P_{\lambda_i}[:, i:]$ 
22:         $P_i[:, i:] = S_i M_{inv}[i, i:]$ 
23:      if  $\lambda_i \neq 0$  then
24:         $P_i[:, i:] \leftarrow {}^iX_{\lambda_i} P_{\lambda_i}[:, i:]$ 

```

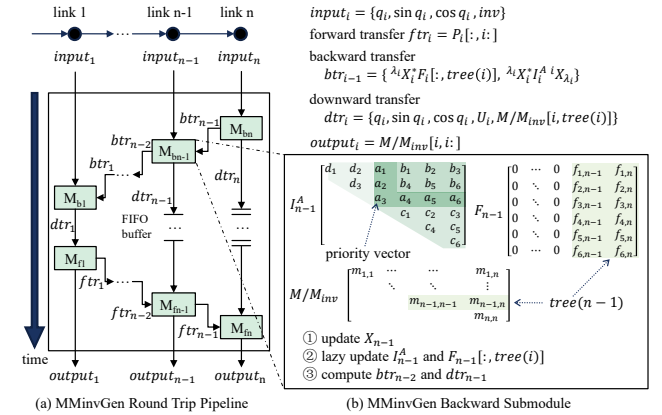


Figure 8: RTP Design for MMinvGen

Compared with the original algorithms, we avoid a whole forward loop, which can greatly reduce hardware resource consumption. Compared with the CRBA algorithm, our algorithm greatly improves the parallelism of computation. We can choose whether to generate the mass matrix through the out_M parameter, and whether to do matrix factorization and inversion then generate the inverse of the mass matrix through the out_{Minv} parameter. The intermediate variables I_i^A and F_i are initialized to all-zero matrices. The notation $tree(i)$ represents the set of id of all nodes contained in the subtree of node i , and $tree_e(i) = tree(i) \setminus i$.

Based on Algorithm 2, we design the MMinvGen RTP, as shown in Fig. 8. The overall structure of the MMinvGen RTP is basically

similar to that of the RNEA RTP, but the direction of the dataflow is reversed. It uses all the optimization methods used in RNEA RTP and Δ RNEA RTP, such as the Incremental Calculation and the Lazy Update for the matrix F_i , and the Sparsity Optimization for the symmetric matrices M , M^{-1} and I_i^A , as show in Fig. 8b.

In addition, it has the following optimization methods:

4.2.1 Priority Vector. The symmetric matrix I_i^A needs to be updated lazily in each submodule, yet its computation is in the critical path of the entire pipeline. We no longer divide it into column vectors in the previous way, but divide it into four vectors according to specific priorities, calculate and pass the vectors in the critical path first. Taking the revolute joint as an example, the calculation of the third column element of the symmetric matrix I_i^A is on the critical path, so we set it as the priority vector. The division method is shown in Fig. 8b.

4.2.2 Fixed-point vs Floating-point Reciprocal. Compared with floating-point numbers, fixed-point addition, subtraction and multiplication are very resource-efficient and fast. However, there are reciprocal operations in Algorithm 2 line 5. And fixed-point division is very slow. Therefore, we need to convert fixed-point numbers into floating-point numbers first, and then use the characteristics of floating-point numbers to quickly find the reciprocal [20]. After getting the result, it is converted back to a fixed-point number to participate in subsequent operations. This can greatly improve performance.

5 ARCHITECTURE OF DADU-RBD

The hardware in the previous section is designed for a serial robotic arm with a fixed base, but the actual robot has a more complex structure and does not have a fixed base. Dadu-RBD needs to take this into account. Based on this, the versatility of Dadu-RBD needs to have two aspects, supporting multiple functions and supporting multiple robots. It should be noted that Dadu-RBD's support for multi-robots is not dynamic, but needs to be reconfigured. For a specific model of robot, only once initial configuration is required. In this section, we discuss how Dadu-RBD optimizes performance and resource usage while implementing a general robot rigid body dynamics accelerator.

5.1 Multifunction Optimization

As mentioned in Section 3.1, thanks to the close relationships among the robot rigid body dynamics calculations, we can fully reuse hardware resources for different robot rigid body dynamics functions.

It is worth noting that in order to completely calculate the result of Δ FD, we must go through 6 steps (① to ⑥), as shown in Fig. 9a. We found that the functions ID, FD, Minv, Δ ID and Δ FD are actually a subset of these 6 calculation steps. Therefore we can implement these functions through a combination of these 6 steps. By implementing these calculation steps and using some auxiliary modules, Dadu-RBD can support all the functions in Table 1. We will cover this in more detail in Section 5.4.

For RNEA and Δ RNEA RTP, the intermediate data v, a, f need to be transferred between their submodules. We further optimized the data paths between them, as shown in Fig. 9b. We name this

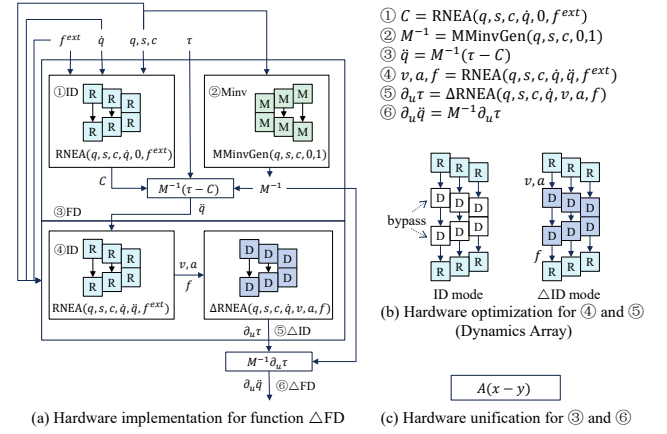


Figure 9: Hardware optimization for multifunction.

submodules' structure the Dynamics Array. It interleaves the submodules of RNEA and Δ RNEA together. Therefore, the data from each submodule of RNEA RTP can be directly transferred to the corresponding submodule of Δ RNEA RTP. The Dynamics Array's submodules can take turns reading data in order, and perform calculations along the array in the form of a pipeline, and then output them in turn. This is similar to the behavior of a systolic array. If there is no need to compute the derivatives, the Δ RNEA submodules can be switched to a data pass mode to support standalone operation of the RNEA.

We can further find that both RNEA and matrix multiplication are used twice. We can further reuse the hardware to perform these two calculations. The format of matrix multiplication is unified as shown in Fig. 9c. A is a symmetric matrix that can be optimized for sparsity.

5.2 Architecture Overview

Dadu-RBD needs to be configured according to the model and parameters of the robot before calculation. To demonstrate the architecture of Dadu-RBD, we use quadruped robot with an arm (Fig. 3) as an example to configure the accelerator. It is a quadruped robot with a mechanical arm, which has $N_B = 19$ links, and the DOF of the robot is $N = 24$ (including the 6-DOF floating base). After the configuration, the structure of Dadu-RBD will be fixed, as shown in Fig. 10.

The inputs of Dadu-RBD include $type, q, \dot{q}, \ddot{q}/\tau, f^{ext}$ and M^{-1} . Parameter $type$ indicates which function Dadu-RBD should run. It also contains some flag informations to provide function options and output types.

The outputs of Dadu-RBD contain $\tau, \ddot{q}, M, M^{-1}, \partial_u \tau$, and $\partial_u \ddot{q}$. They can be selected and combined into the output of any function. The function Δ FD can optionally output M^{-1} .

Dadu-RBD contains 8 modules, namely Decode Module, Encode Module, Global Trigonometric Module, Input Stream Module, Forward-Backward Module, Backward-Forward Module, Schedule Module and Feedback Module. Next, we will briefly introduce the main design methods of the Dadu-RBD architecture.

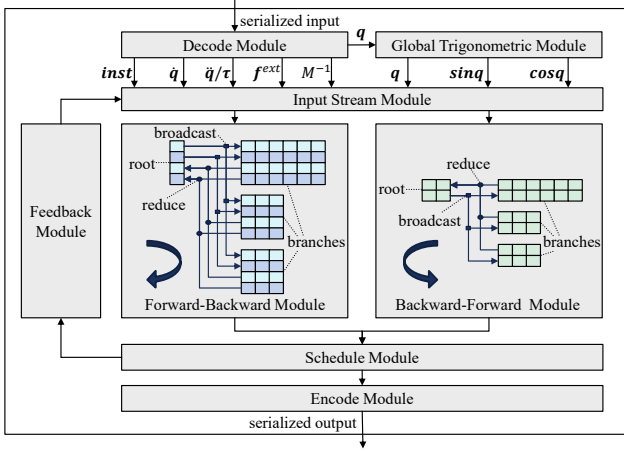


Figure 10: Dadu-RBD architecture.

5.2.1 Decode and Encode. Depending on the chosen function, Dadu-RBD will have different inputs and outputs. In order to facilitate the design of the multifunctional pipeline, we unify the formats of all inputs and outputs. The Decode Module will deserialized and decode the data from the input interfaces. The Encode module can convert data into a CPU-friendly type for subsequent use.

5.2.2 Global Trigonometric Module. Most submodules require the values of $\sin q$ and $\cos q$. Therefore we compute all $\sin q$ and $\cos q$ in advance in Global Trigonometric Module. This module computes the Taylor-series expansion of trigonometric functions for fast approximate solutions. With loop unrolling optimizations, the entire module can be pipelined to achieve high-throughput performance.

5.2.3 Scheduling System. The scheduling system of Dadu-RBD consists of Input Stream Module, Schedule Module and Feedback Module. They constitute a state machine with a feedback structure. With the support of this scheduling system, Dadu-RBD can provide timely and accurate data to the multifunctional pipelines, and coordinate conflicts within and between functions.

The Input Stream Module collects the corresponding data according to the micro-instructions $inst$, and then provides data to the specified module in a certain order. There are some subtle differences between the function type $type$ and the micro-instructions $inst$. The $type$ is the function interface argument specified by the upper-layer application, while $inst$ is the microinstruction inside Dadu-RBD. An instruction $type$ will be translated into multiple different micro-instructions $inst$ during its life cycle, so as to facilitate the scheduling of multifunctional pipelines. In this way, modules and submodules can automatically select the output type and path according to the current micro-instructions, thereby realizing the dynamic switching of the dataflow.

The Schedule Module is responsible for integrating and scheduling the calculation results of multifunctional pipelines. It also provides the remaining few vector subtraction and matrix multiplication calculations for the FD, Δ FD and Δ iFD functions. When calculating numerical integration in TO or MPC, Schedule Module can also combine the calculation results of the FD function with the

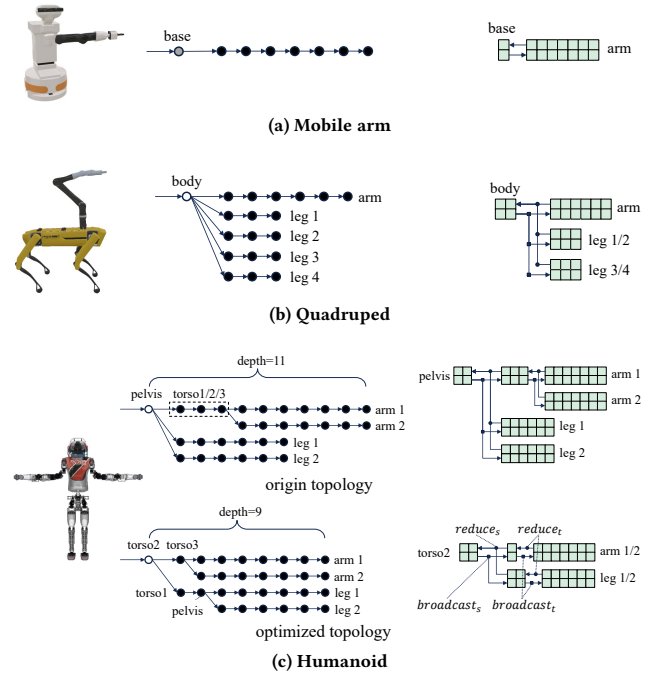


Figure 11: Optimization for submodules' organization.

current state to generate a new integration step. This new task will be passed to the Feedback Module, waiting for the next calculation.

5.2.4 Forward-Backward and Backward-Forward. Forward-Backward and Backward-Forward are the abstraction of the dataflow types in dynamic algorithms. The dataflow of the Forward-Backward module pass forward first, and then pass backward, while the dataflow of the Backward-Forward module is just the opposite. Various dynamics algorithms can be implemented through the combination of these two dataflow, only the calculation logic is different.

The Forward-Backward Module is used to compute RNEA and its gradient Δ RNEA. It has $4N_B$ submodules, namely R_{fi} , R_{bi} , D_{fi} and D_{bi} , as discussed in Section 4.1. The submodules are divided into root and multiple branches according to the structure of the robot. The data between them needs to be broadcast and reduced. Specifically, we need to copy the data for each branch in the forward pass, and sum the data provided by each branch in the backward pass. All submodules in a branch can form a Dynamics Array (Fig. 9b).

In the current implementation, the Backward-Forward Module is used to compute MMinvGen. It has $2N_B + 2$ submodules, namely M_{bi} and M_{fi} , where $i \in [0, N_B]$, and also be divided into root and multiple branches. It has the potential to implement the ABA algorithm, but due to resource constraints we do not currently implement it.

5.3 Structure-Adaptive Pipelines

There are many different structures of robots, such as mobile arm (Tiago, Fig. 11a), quadruped robot (Spot-arm, Fig. 11b), and humanoid robot (Atlas, Fig. 11c), etc. These different robot structures

will change the dependencies between iterations of the dynamics algorithms. Therefore, the pipeline structure needs to be adjusted according to the robot structure. So we call it Structure-Adaptive Pipelines.

5.3.1 Organization of Submodules. We design a general optimization method that adapts the organization of Dadu-RBD submodules according to the robot structure. We illustrate this approach with the three robot examples above, as shown in Fig. 11. The left side of each subfigure is the topological relationship of the robot structure, and the right side is the organizational relationship of the submodules in Backward-Forward Module. The organizational relationship of the submodules in Forward-Backward Module is the same as that of Backward-Forward Module.

Tiago has a 3-DOF mobile base and a 7-DOF arm, its topology is linear. So we can just use a root and a branch to organize all submodules. The root joint type is planar, which only has 3 degrees of freedom, so it does not need to be split into two joints. As shown in Fig. 11a, the root corresponds to the base link, and the branch corresponds to the robotic arm.

Spot-arm has a 6-DOF body, four 3-DOF legs and a 6-DOF arm. It has a tree topology. We can organize all submodules into a root and three branches. As shown in Fig. 11b, the root corresponds to the body link, a 6-size branch corresponds to the robotic arm, and two 3-size branches correspond to the four legs. Here we optimize the number of leg branches. Since the legs of the Spot are all symmetrical, the sparsity of the leg parameters is the same, and only a few parameters differ, most of which differ only in sign. According to this observation, we can handle two symmetrical legs with one 3-size branch. This saves hardware resources. At the same time, due to the small topological depth of the leg branches, the complexity of calculating the leg dynamics is much smaller. Therefore, the delay of each submodule will be much smaller, and the processing time of one robotic arm can fully support the processing of two legs.

Atlas is a humanoid robot. It has a body with a waist, two arms and two legs, forming a topological tree. The waist is made up of three joints (torso1, torso2 and torso3) that connect the torso to the pelvis. This makes it impossible for branching structures to exist only at the root. Traditionally, people will define the pelvis as the root, as shown in Fig. 11c. Under this definition, the depth of the topological tree will become 11. As we have discussed before, the complexity of submodules will increase as the iteration depth increases (Section 4.1.4). So we can optimize Dadu-RBD by adjusting the depth of robot topology tree, as shown in Fig. 11c. We redefine torso2 as the root of the robot. This does not change the robot's topological connectivity, but the depth of the topological tree was reduced to 9, and the depth of each branch is balanced. This can reduce a lot of resource consumption, while reducing computing latency. Because Atlas is a symmetrical robot, we can also handle two arms or two legs with a single branch array.

It is worth noting that in Fig. 11c, the $broadcast_s$ and $reduce_s$ connected to the root (torso2) submodules are oriented to different hardware branches, while the $broadcast_t$ and $reduce_t$ connected to torso3 or pelvis submodules are oriented to the same hardware branches, which are time-division multiplexed. This difference also exists in the submodules' organization of the Spot-arm robot.

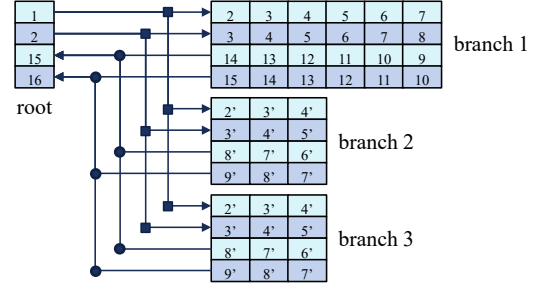


Figure 12: Pipeline Stages for SAPs.

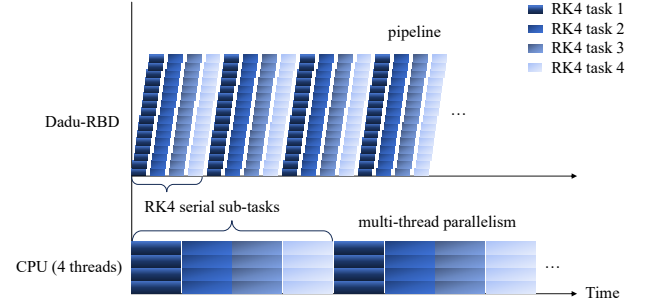


Figure 13: Scheduling method for Dadu-RBD and CPU when processing partially parallelizable tasks. The blocks with the same color are independent tasks.

5.3.2 Pipeline Stages. We do not perform fine-grained pipelining for the components of matrix operations, or the resource consumption would be very large. We do not only perform coarse-grained pipelining for each module, either. Such fine-grained pipelines combined with coarse-grained pipelines are precisely the architecture proposed in the previous work [34]. Instead, Dadu-RBD disassembles the algorithm modules into submodules, and then performs medium-grained pipelining among these submodules.

Fig. 12 shows the pipeline stages for the Forward-Backward Module. The pipeline of Backward-Forward Module is similar to this. Each branch array is independent of each other, so they can perform asynchronous parallel computing. As the complexity of submodules will increase with the increase of iteration depth (Section 4.1.4), the pipeline cycle of branch 1 is almost double that of branch 2/3. So branch 2/3 can handle twice as many tasks as branch 1.

5.3.3 Pipeline Schedule. In many TO and MPC algorithms, computations on multiple sampling points can be processed in parallel. We can easily use SPAs to speed up these tasks without worrying about dependencies. But sometimes we need to call the dynamics function serially. For example, the 4th-order Runge-Kutta sensitivity analysis has 4 serial sub-tasks on each sampling point. These tasks need to be scheduled appropriately, as shown in Fig. 13. Subsequent sub-tasks need to be scheduled after the predecessor tasks are completed. Before that, Dadu-RBD can compute other independent batched tasks first. Under this scheduling strategy, Dadu-RBD can effectively avoid the negative impact of serial sub-tasks on parallelism. For comparison, we also plot how the multithreaded CPU schedules these tasks. The CPU performs spatial parallelism

through multi-core resources, while Dadu-RBD performs temporal parallelism through SAPs.

5.3.4 Branch-induced Sparsity. In the dynamics algorithms mentioned in this article, the structure of the robot determines the dependencies between variables (λ_i). Therefore, there will be no direct dependencies between submodules of different branches. According to this feature, submodules can only keep relevant matrix columns instead of the entire parameter matrix. This is robot branch-induced sparsity.

5.3.5 Root Submodules. In order to avoid complicated calculations, the base link is split into two parts. The robot assumed in the architecture diagram has a floating base with 6 DOF. We can split the 6-DOF virtual joint into two 3-DOF virtual joints, which are spherical joint and 3-DOF translation joint. This reduces the computational complexity of the root node.

According to the different needs of the application, Dadu-RBD can provide different modes to handle the dynamics for the root. For the virtual joint corresponding to the base link, we can treat it as an ordinary joint and use the standard algorithms for calculation. For some bases that are almost not affected by dynamics (for example a robotic arm attached to a very heavy base), we can just provide a state for the base link from the input as an initialization for the following links, or even ignore the root node directly. Both Forward-Backward Module and Backward-Forward Module can select the mode of the root submodules through the micro-instructions *inst*. This can improve the versatility of the Dadu-RBD accelerator and reduce unnecessary calculations in some cases.

5.4 Dataflow

All the modules and submodules described above are completely driven by data, and the data passed between them are implemented by FIFO streams. Therefore, these modules and submodules can be organized in the form of dataflow.

As shown in Fig. 14, the dataflow path and activated modules (submodules) for different functions in Table 1 are marked in dark color. For each function, the Decode Module, Global Trigonometric Module, Input Stream Module, Schedule Module and Encode Module all need to be activated.

For functions ID, FD, M, Minv, Δ ID and Δ iFD, their dataflow is relatively simple, which is a combination of some modules and submodules of Dadu-RBD. While for the function Δ FD, the dataflow is complex, especially when batch tasks are calculated at the same time. The Δ FD function has three stages, the first stage is to calculate the FD function, the second stage is to calculate the Δ ID function, and the third stage is to calculate the final result. Both the first and second stages use the Forward-Backward Module for calculations, so we need the Feedback Modules to write data back to the Input Stream Module. Between stages, the Feedback Module is also responsible for saving necessary intermediate results for a second use, such as the inverse of the mass matrix. These different stages are distinguished by the micro-instructions *inst*, so that even with multiple tasks, the overall dataflow will not be messed up.

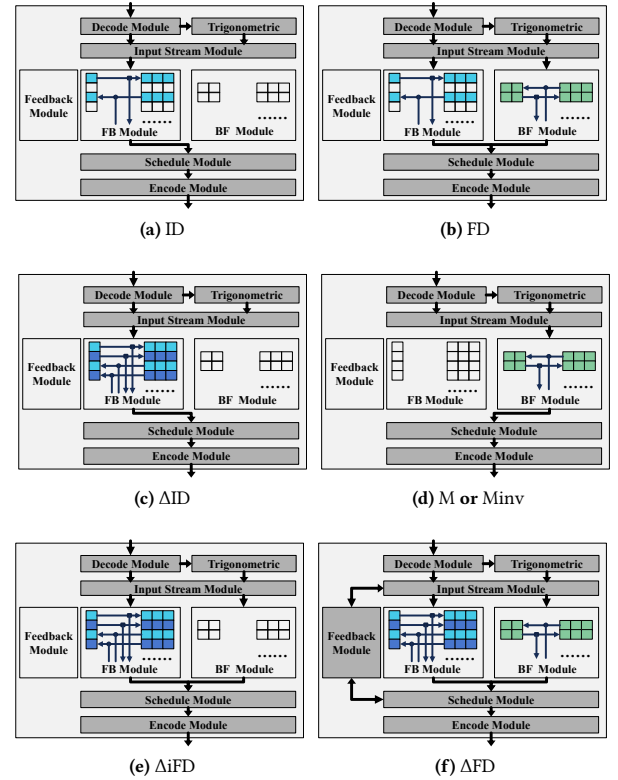


Figure 14: Dataflow of different functions.

6 EVALUATION

We implement and evaluate the Dadu-RBD architecture on the Xilinx platform. Vitis and Vivado are used to synthesize the architecture, simulate the dataflow and estimate the power. To compare with the widely-used C++ dynamics library Pinocchio [6] and previous state-of-the-art CPU[40], GPU[41] and FPGA[34, 40] works, we use the same FPGA chip as Robomorphic. The hardware configurations used in the following evaluations are listed in Table 2. Among them, Jetson AGX Orin is NVIDIA's high-performance edge computing module, with 12-core Arm® Cortex®-A78AE CPU and 2048-core NVIDIA Ampere architecture GPU. We will use its performance as a baseline for performance evaluation under MAXN Power mode (about 60W). In addition, the Core i9-13900HX and RTX 4090 Mobile represent the highest performance levels for mobile CPUs and mobile GPUs, respectively. The CPU runtime power can reach up to 140W level, and the GPU can reach up to 175W level. They are paired with dual-channel 5600MHz DDR5 memory. We will also use their performance as a reference. We perform the best compilation optimization options (-O3 and SIMD support) on the CPU and GPU libraries.

We use the Dadu-RBD architecture to realize the dynamics functions for three robots, LBR iiwa [1], HyQ [44] and Atlas [9], respectively. This is consistent with two previous works [6, 41]. FPGA memory interface bandwidth is limited to a maximum of 32GB/s. All evaluations take the I/O time into account. The input

Table 2: Hardware Configurations in Evaluations.

Type	Processor	Freq	Usage
CPU	AGX Orin	2.2G	Evaluate [6]
CPU	i9-13900HX	5.4G	Evaluate [6]
GPU	AGX Orin	1.3G	Evaluate [41]
GPU	RTX 4090M	1.8G	Evaluate [41]
FPGA	XVCU9P	56M	Used in [34, 40]
FPGA	XVCU9P	125M	Evaluate Dadu-RBD

and output of Dadu-RBD are in the form of data streams, so the I/O overhead of Dadu-RBD can be greatly masked. All the results are the averages of millions of runs.

After experimentation and tuning, Dadu-RBD is able to run at 125MHz in the target FPGA. Under this condition, the performance and energy consumption reach a balance.

6.1 Latency and Throughput

We compared the latency with CPUs, and compared the throughput with CPUs/GPUs. The evaluation method for latency is running 128 different tasks with a single thread, repeated 10k times, and took the average running time of each task. Such a task load is to accord with the actual situation. The evaluation method for throughput is running 256 batched tasks with multiple threads (12 threads for Jetson AGX Orin, 32 threads for i9-13900HX, grids/blocks/threads for gpu is automatically allocated by library functions), repeated 10k times, and took the average running time of each batched tasks to get the throughput. 256 batched tasks is selected because (1) the highest batch size in previous GPU benchmarks was 256 [41]; (2) Referring to the algorithm paper of MPC [11], the time horizon of MPC is about 1 second. Considering that the frequency requirement of MPC is 100Hz, the corresponding time step is 0.01 second, so the total sampling points are about 100. Here we set the number of batches to 256 to cover most of the current needs. Of course, we will discuss the impact of larger batch sizes on performance later.

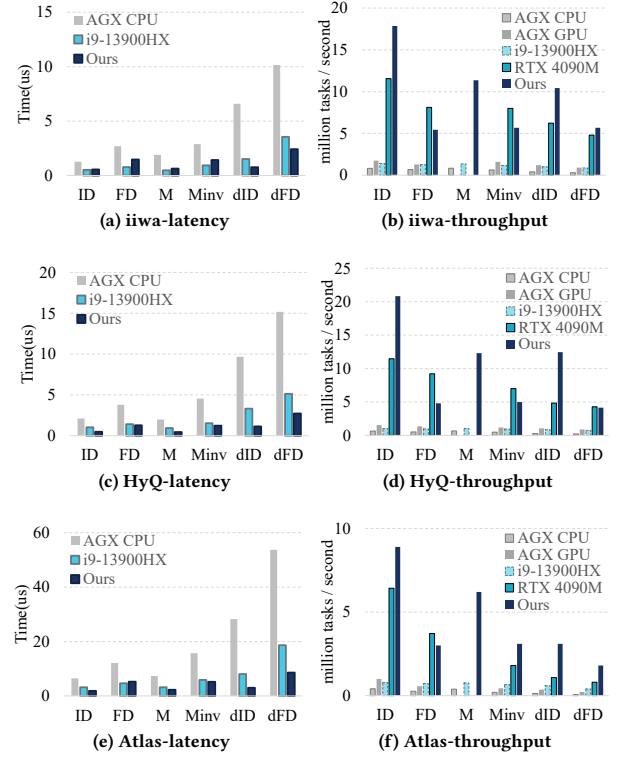
The performance comparison between Dadu-RBD and CPU, GPU libraries [6, 41] are shown in Fig. 15. For all supported functions, the latencies of Dadu-RBD are better than that of the CPU in Jetson AGX Orin. When comparing the latency with i9-13900HX, the implementation of Dadu-RBD on the current chip is mostly better than i9, but there are also some functions that are not as good as i9.

Latencies (lower is better) on different functions against (1) AGX CPU: 0.12×-0.55×, average 0.29×; (2) i9-13900HX: 0.34×-1.91×, average 0.82×.

For GPU, GRiD [41] does not realize the calculation of the mass matrix, so there is no data in the corresponding position in the figure. The throughput of Dadu-RBD's implementation is much better than that of the AGX CPU, AGX GPU and i9-13900HX. But in some functions, it cannot win RTX 4090M.

Throughput (higher is better) on different functions against (1) AGX CPU: 8.1×-43.6×, average 19.2×; (2) AGX GPU: 3.5×-13.4×, average 7.2×; (3) i9-13900HX: 4.1×-20.2×, average 8.2×; (4) RTX 4090M: 0.5×-2.8×, average 1.4×.

The acceleration effect of Dadu-RBD on functions Minv and FD is not very good, mainly because the inverse of the matrix is really

**Figure 15: Latency and throughput comparisons with Pinocchio [6] on CPU and GRiD [41] on GPU.**

unfriendly to FPGA. And in order to save resources, we did not use the most efficient ABA algorithm to calculate FD.

Dadu-RBD is not specifically optimized for latency. Instead, it is designed to maximize the computing throughput. So compared with the previous work Robomorphic[34], our computational latency is increased a little bit for the same robot using the same FPGA chip. Specifically, the latency of computing Δ FD for robot iiwa is $0.76\mu s$ in Dadu-RBD, while the latency in Robomorphic is $0.61\mu s$. This is mainly due to the additional data transfer overhead caused by the dataflow in Dadu-RBD for supporting multiple functions and the SAPs.

On the other hand, we compare the throughput with the previous works [34, 40], as shown in Fig. 16. (The data is collected from [40].) They only implemented the function Δ FD with CPU, GPU and FPGA. As a comparison, when running batched tasks, Dadu-RBD further gives 10.3×-13× performance improvement over their optimized CPU implementation, 3.4×-11.3× improvement over their GPU's, and 6.3×-7.0× improvement over their FPGA's.

Because the GPU has a strong parallel capability, we also did a larger scale parallel test, as shown in Fig. 17. It can be seen that the best fit of GPUs is >1024 batch size. The calculation time will increase proportionally and linearly after the size of batches continues to increase. RTX 4090M will outperform our implementation when batch size is more than 512. Dadu-RBD improves throughput through RTP and SAPs, so the throughput will not fluctuate too

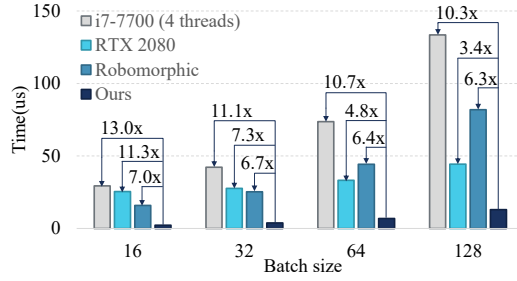


Figure 16: Batched ΔiFD tasks compared with CPU, GPU and FPGA.

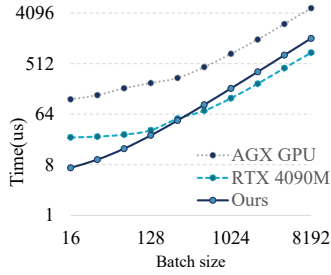


Figure 17: Batched ΔFD tasks compared with GPUs.

much after the pipeline is saturated. If we want to further improve throughput, we can instantiate multiple SAPs.

6.2 End-to-End Application

In the robot application mentioned in Fig. 2, there are three kind of tasks can be accelerated by Dadu-RBD, namely **forward dynamics**, **inverse of the mass matrix** and **derivatives of dynamics**, which can be calculated by functions FD and ΔFD . With the help of the RTP and SAPs, Dadu-RBD can achieve 11.2 $\times$ performance speedup on these tasks. When Dadu-RBD accelerates the supported tasks, the CPU can compute other batch tasks at the same time. Compared with a 4-thread CPU implementation, Dadu-RBD can help the entire system increase the control frequency by 80%.

6.3 Resource Usage, Power and Energy

Because each submodule needs to perform computation independently, Dadu-RBD requires many computing resources. But compared with the previous work [34], our architecture is able to achieve higher performance with similar resource usage. Our implementation takes up 62% DSP, 17% FF and 54% LUT of the target FPGA. The previous work mentioned that due to the lack of DSP, they could not instantiate a new independent unit on the same FPGA chip to parallelize the computation, so they also take up at least half of the DSP.

The power dissipation of our accelerator varies for different functions and robot configurations. We estimate Dadu-RBD's power for LBR iiwa. For different functions the power dissipation ranges from 6.2W to 36.8W. As a comparison with previous work Robomorphic [34] with power 9.6W, which only supports ΔiFD function, Dadu-RBD's power for function ΔiFD is up to 31.2W. With 3.25 $\times$ higher power, Dadu-RBD's speed is 6.6 $\times$ faster than that of Robomorphic, so the energy consumption of Robomorphic is 2.0 $\times$ higher

than Dadu-RBD's. In terms of energy delay product, Dadu-RBD is 13.2 $\times$ better than Robomorphic.

7 CONCLUSION

In this paper, by analyzing robot planning and control algorithms, we learned that the calculation of robot dynamics and its derivatives is a basic and important part, and there are often performance bottlenecks. Many of these dynamics functions can be accelerated through our proposed Dadu-RBD architecture. By designing RTP/SAPs, the performance and throughput are greatly improved, while keeping resource consumption low. The key to the whole work is that we have effectively co-optimized the hardware architecture with the dynamics algorithms and the robot topology structure. It is worth noting that there is an independent and concurrent work published on ISCA 2023 [33], which also has similar ideas. This idea can also be used in other fields of robotics, waiting for everyone to explore together.

ACKNOWLEDGMENTS

This work was supported by National Key R&D Program of China (No. 2018YFA0701500), by National Natural Science Foundation of China (Nos. 62122076, 61834006, and 62025404), by Key Research Program of Frontier Sciences, CAS (No. ZDBS-LY-JSC012), and by Strategic Priority Research Program of CAS (No. XDB44000000).

REFERENCES

- [1] KUKA AG. 2022. LBR iiwa. <https://www.kuka.com/products/robotics-systems/industrial-robots/lbr-iiwa>
- [2] Thomas Antony and Michael J. Grant. 2017. Rapid Indirect Trajectory Optimization on Highly Parallel Computing Architectures. *Journal of Spacecraft and Rockets* 54, 5 (2017), 1081–1091. <https://doi.org/10.2514/1.A33755>
- [3] Nitin Arora, Ryan P Russell, and Richard W Vuduc. 2009. Fast sensitivity computations for trajectory optimization. *Advances in the Astronautical Sciences* 135, 1 (2009), 545–560.
- [4] Mohammad Bakhshalipour, Seyed Borna Ehsani, Mohamad Qadri, Dominic Guri, Maxim Likhachev, and Phillip B Gibbons. 2022. Racod: algorithm/hardware co-design for mobile robot path planning. In *Proceedings of the 49th Annual International Symposium on Computer Architecture*. 597–609.
- [5] Justin Carpentier. 2018. Analytical Inverse of the Joint Space Inertia Matrix. <https://hal.laas.fr/hal-01790934>
- [6] Justin Carpentier and Nicolas Mansard. 2018. Analytical Derivatives of Rigid Body Dynamics Algorithms. In *Robotics: Science and Systems (RSS 2018)*.
- [7] Ewen Dantec, Rohan Budhiraja, Adria Roig, Teguh Lembono, Guilhem Saurel, Olivier Stasse, Pierre Fernbach, Steve Tonneau, Sethu Vijayakumar, Sylvain Calinon, Michel Taix, and Nicolas Mansard. 2021. Whole Body Model Predictive Control with a Memory of Motion: Experiments on a Torque-Controlled Talos. In *2021 IEEE International Conference on Robotics and Automation (ICRA)*. 8202–8208. <https://doi.org/10.1109/ICRA48506.2021.9560742>
- [8] Moritz Diehl, Hans Georg Bock, Holger Diedam, and P-B Wieber. 2006. Fast direct multiple shooting algorithms for optimal robot control. *Fast motions in biomechanics and robotics: optimization and feedback control* (2006), 65–93.
- [9] Boston Dynamics. 2022. Atlas. <https://www.bostondynamics.com/atlas>
- [10] Farbod Farshidian. 2018. Optimal Control for Switched Systems(OCS2). <https://github.com/leggedrobotics/ocs2>
- [11] Farbod Farshidian, Edo Jelavic, Asutosh Satapathy, Markus Gifftthaler, and Jonas Buchli. 2017. Real-time motion planning of legged robots: A model predictive control approach. In *2017 IEEE-RAS 17th International Conference on Humanoid Robotics (Humanoids)*. 577–584. <https://doi.org/10.1109/HUMANOIDS.2017.8246930>
- [12] Roy Featherstone. 2008. *Rigid Body Dynamics Algorithms*. Springer US, Boston, MA. <https://doi.org/10.1007/978-1-4899-7560-7>
- [13] Markus Gifftthaler, Michael Neunert, Markus Stäuble, and Jonas Buchli. 2018. The control toolbox — An open-source C++ library for robotics, optimal and model predictive control. In *2018 IEEE International Conference on Simulation, Modeling, and Programming for Autonomous Robots (SIMPAN)*. 123–129. <https://doi.org/10.1109/SIMPAN.2018.8376281>
- [14] Markus Gifftthaler, Michael Neunert, Markus Stäuble, Jonas Buchli, and Moritz Diehl. 2018. A Family of Iterative Gauss-Newton Shooting Methods for Nonlinear

- Optimal Control. In *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 1–9. <https://doi.org/10.1109/IROS.2018.8593840>
- [15] Knut Graichen and Bartosz K pernick. 2012. *A real-time gradient method for nonlinear model predictive control*. INTECH Open Access Publisher London.
 - [16] Dipanwita Guhathakurta, Fatemeh Rastgar, M. Aditya Sharma, K. Madhava Krishna, and Arun Kumar Singh. 2022. Fast Joint Multi-Robot Trajectory Optimization by GPU Accelerated Batch Solution of Distributed Sub-Problems. *Frontiers in Robotics and AI* 9 (2022).
 - [17] Yinhe Han, Yuxin Yang, Xiaoming Chen, and Shiqi Lian. 2020. DaDu Series - Fast and Efficient Robot Accelerators. In *2020 IEEE/ACM International Conference On Computer Aided Design (ICCAD)*. 1–8.
 - [18] Steffen Heinrich, Andr  Zoufahl, and Ra l Rojas. 2015. Real-time trajectory optimization under motion uncertainty using a GPU. In *2015 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 3572–3577. <https://doi.org/10.1109/IROS.2015.7353876>
 - [19] Ayonga Hereid, Omar Harib, Ross Hartley, Yukai Gong, and Jessie W. Grizzle. 2019. Rapid Trajectory Optimization Using C-FROST with Illustration on a Cassie-Series Dynamic Walking Biped. arXiv:1807.06614 (Mar 2019). <http://arxiv.org/abs/1807.06614> arXiv:1807.06614 [cs].
 - [20] Matei Ist an and Bogdan Pasca. 2015. Fixed-Point Implementations of the Reciprocal, Square Root and Reciprocal Square Root Functions. (Nov 2015). <https://doi.org/10.1109/ARITH.2017.31>
 - [21] Tianyu Jia, En-Yu Yang, Yu-Shun Hsiao, Jonathan Cruz, David Brooks, Gu-Yeon Wei, and Vijay Janapa Reddi. 2022. OMu: A Probabilistic 3D Occupancy Mapping Accelerator for Real-time OctoMap at the Edge. Number arXiv:2205.03325. <https://doi.org/10.48550/arXiv.2205.03325> arXiv:2205.03325 [cs] type: article.
 - [22] S. Katayama and T. Ohtsuka. 2020. Efficient Solution Method Based on Inverse Dynamics of Optimal Control Problems for Fixed-Based Rigid-Body Systems. *IFAC-PapersOnLine* 53, 2 (2020), 6483–6489. <https://doi.org/10.1016/j.ifacol.2020.12.1794>
 - [23] Bulat Khusainov, Eric Kerrigan, Andrea Suardi, and George Constantinides. 2018. Nonlinear predictive control on a heterogeneous computing platform. *Control Engineering Practice* 78 (2018), 105–115. <https://doi.org/10.1016/j.conengprac.2018.06.016>
 - [24] Twan Koolen and Robin Deits. 2019. Julia for robotics: simulation and real-time control in a high-level programming language. In *2019 International Conference on Robotics and Automation (ICRA)*. 604–611. <https://doi.org/10.1109/ICRA.2019.8793875>
 - [25] Dimitris Kouzoupis, Rien Quirynen, Boris Houska, and Moritz Diehl. 2016. A block based ALADIN scheme for highly parallelizable direct Optimal Control. In *2016 American Control Conference (ACC)*. 1124–1129. <https://doi.org/10.1109/ACC.2016.7525066>
 - [26] Aravindh Krishnamoorthy and Deepak Menon. 2013. Matrix inversion using Cholesky decomposition. In *2013 Signal Processing: Algorithms, Architectures, Arrangements, and Applications (SPA)*. 70–72.
 - [27] Bartosz K pernick, Sebastian S  , Endric Schubert, and Knut Graichen. 2014. A synthesis strategy for nonlinear model predictive controller on FPGA. In *2014 UKACC International Conference on Control (CONTROL)*. 662–667. <https://doi.org/10.1109/CONTROL.2014.6915218>
 - [28] Peter Zhi Xuan Li. 2020. *High-throughput computation of shannon mutual information on chip*. Ph. D. Dissertation. Massachusetts Institute of Technology.
 - [29] Kevin M. Lynch and Frank C. Park. 2017. *Modern robotics: mechanics, planning, and control*. Cambridge University Press, Cambridge, UK.
 - [30] Carlos Mastalli, Rohan Budhiraja, Wolfgang Merkt, Guilhem Saurel, Bilal Hammoud, Maximilien Naveau, Justin Carpentier, Ludovic Righetti, Sethu Vijayakumar, and Nicolas Mansard. 2020. Crocodyl: An Efficient and Versatile Framework for Multi-Contact Optimal Control. In *2020 IEEE International Conference on Robotics and Automation (ICRA)*. 2536–2542. <https://doi.org/10.1109/ICRA40945.2020.9196673>
 - [31] Ian McInerney, George A. Constantinides, and Eric C. Kerrigan. 2018. A Survey of the Implementation of Linear Model Predictive Control on FPGAs. *IFAC-PapersOnLine* 51, 20 (2018), 381–387. <https://doi.org/10.1016/j.ifacol.2018.11.063>
 - [32] Sean Murray, William Floyd-Jones, Ying Qi, George Konidaris, and Daniel J. Sorin. 2016. The microarchitecture of a real-time robot motion planning accelerator. In *2016 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. 1–12. <https://doi.org/10.1109/MICRO.2016.7783748>
 - [33] Sabrina M Neuman, Radhika Ghosal, Thomas Bourgeat, Brian Plancher, and Vijay Janapa Reddi. 2023. RoboShape: Using Topology Patterns to Scalably and Flexibly Deploy Accelerators Across Robots. In *Proceedings of the 50th Annual International Symposium on Computer Architecture*. 1–13.
 - [34] Sabrina M. Neuman, Brian Plancher, Thomas Bourgeat, Thierry Tambe, Srinivas Devadas, and Vijay Janapa Reddi. 2021. Robomorphic Computing: A Design Methodology for Domain-Specific Accelerators Parameterized by Robot Morphology. In *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*. 674–686. <https://doi.org/10.1145/3445814.3446746>
 - [35] Michael Neunert, C dric de Crousaz, Fadri Furrer, Mina Kamel, Farbod Farshidian, Roland Siegwart, and Jonas Buchli. 2016. Fast nonlinear Model Predictive Control for unified trajectory optimization and tracking. In *2016 IEEE International Conference on Robotics and Automation (ICRA)*. 1398–1404. <https://doi.org/10.1109/ICRA.2016.7487274>
 - [36] Michael Neunert, Markus St uble, Markus Giffthaler, Carmine D Bellicoso, Jan Carius, Christian Gehring, Marco Hutter, and Jonas Buchli. 2018. Whole-body nonlinear model predictive control through contacts for quadrupeds. *IEEE Robotics and Automation Letters* 3, 3 (2018), 1458–1465.
 - [37] Zherong Pan, Bo Ren, and Dinesh Manocha. 2019. GPU-Based Contact-Aware Trajectory Optimization Using a Smooth Force Model. In *Proceedings of the 18th Annual ACM SIGGRAPH/Eurographics Symposium on Computer Animation*. <https://doi.org/10.1145/3309486.3340246>
 - [38] Helfried Peyrl, Hans Joachim Ferreau, and Dimitris Kouzoupis. 2015. A Hybrid Hardware Implementation for Nonlinear Model Predictive Control. *IFAC-PapersOnLine* 48, 23 (Jan 2015), 87–93. <https://doi.org/10.1016/j.ifacol.2015.11.266>
 - [39] Brian Plancher and Scott Kuindersma. 2020. *A Performance Analysis of Parallel Differential Dynamic Programming on a GPU*. Springer Proceedings in Advanced Robotics, Vol. 14. Springer International Publishing, Cham, 656–672. https://doi.org/10.1007/978-3-030-44051-0_38
 - [40] Brian Plancher, Sabrina M. Neuman, Thomas Bourgeat, Scott Kuindersma, Srinivas Devadas, and Vijay Janapa Reddi. 2021. Accelerating Robot Dynamics Gradients on a CPU, GPU, and FPGA. *IEEE Robotics and Automation Letters* 6, 2 (2021), 2335–2342. <https://doi.org/10.1109/LRA.2021.3057845>
 - [41] Brian Plancher, Sabrina M. Neuman, Radhika Ghosal, Scott Kuindersma, and Vijay Janapa Reddi. 2022. GRiD: GPU-Accelerated Rigid Body Dynamics with Analytical Gradients. In *2022 International Conference on Robotics and Automation (ICRA)*. 6253–6260. <https://doi.org/10.1109/ICRA46639.2022.9812384>
 - [42] Ludovic Righetti, Jonas Buchli, Michael Mistry, Mrinal Kalakrishnan, and Stefan Schaal. 2013. Optimal distribution of contact forces with inverse-dynamics control. *The International Journal of Robotics Research* 32, 3 (2013), 280–298. <https://doi.org/10.1177/0278364912469821>
 - [43] Jacob Sacks, Divya Mahajan, Richard C. Lawson, and Hadi Esmaeilzadeh. 2018. RoboX: An End-to-End Solution to Accelerate Autonomous Control in Robotics. In *2018 ACM/IEEE 45th Annual International Symposium on Computer Architecture (ISCA)*. 479–490. <https://doi.org/10.1109/ISCA.2018.00047>
 - [44] C Semini, N G Tsarakakis, E Guglielmino, M Focchi, F Cannella, and D G Caldwell. 2011. Design of HyQ - a hydraulically and electrically actuated quadruped robot. *Proceedings of the Institution of Mechanical Engineers, Part I: Journal of Systems and Control Engineering* 225, 6 (2011), 831–849. <https://doi.org/10.1177/0959651811402275>
 - [45] M.W. Spong and R. Ortega. 1990. On adaptive inverse dynamics control of rigid robots. *IEEE Trans. Automat. Control* 35, 1 (1990), 92–95. <https://doi.org/10.1109/9.45152>
 - [46] Yuval Tassa, Tom Erez, and Emanuel Todorov. 2012. Synthesis and stabilization of complex behaviors through online trajectory optimization. In *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*. 4906–4913. <https://doi.org/10.1109/IROS.2012.6386025>
 - [47] R Tedrake. 2019. Drake: Model-based design and verification for robotics. <https://drake.mit.edu>
 - [48] Webots. [n. d.]. <http://www.cyberbotics.com>. <http://www.cyberbotics.com> Open-source Mobile Robot Simulation Software.
 - [49] Fang Xu, Hong Chen, Xun Gong, and Qin Mei. 2016. Fast Nonlinear Model Predictive Control on FPGA Using Particle Swarm Optimization. *IEEE Transactions on Industrial Electronics* 63, 1 (2016), 310–321. <https://doi.org/10.1109/TIE.2015.2464171>
 - [50] Fang Xu, Hong Chen, Weiwei Jin, and Yueting Xu. 2014. FPGA implementation of nonlinear model predictive control. In *The 26th Chinese Control and Decision Conference (2014 CCDC)*. 108–113. <https://doi.org/10.1109/CCDC.2014.6852127>
 - [51] Yuxin Yang, Shiqi Lian, Xiaoming Chen, and Yinhe Han. 2020. Accelerating RRT Motion Planning Using TCAM. In *Proceedings of the 2020 on Great Lakes Symposium on VLSI*. Association for Computing Machinery, 481–486. <https://doi.org/10.1145/3386263.3406948>
 - [52] Liwei Zhou, Matthew Jahnes, and Matthias Preindl. 2022. Modular Model Predictive Control of a 15-kW, Kilo-to-Mega-Hertz Variable-Frequency Critical-Soft-Switching Nonisolated Grid-Tied Inverter With High Efficiency. *IEEE Transactions on Power Electronics* 37, 10 (2022), 12591–12605. <https://doi.org/10.1109/TPEL.2022.3178417>